Маніпулювання даними за допомогою Pandas 🐼

Базові структури Pandas

In [1]:

```python
import pandas as pd
import numpy as np
```

In [2]:

```python
data = [10,20,30,40]
s1 = pd.Series(data)

print(s1)
```

```
0    10
1    20
2    30
3    40
dtype: int64
```

In [3]:

```python
data_dict = {'Apple': 1.5, 'Banana': 0.5, 'Cherry': 3.0}
s2 = pd.Series(data_dict)

print(s2)
```

```
Apple     1.5
Banana    0.5
Cherry    3.0
dtype: float64
```

In [4]:

```python
s3 = pd.Series(5, index=['a', 'b', 'c'])

print(s3)
```

```
a    5
b    5
c    5
dtype: int64
```

In [5]:

```python
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'London', 'Paris']
}
df1 = pd.DataFrame(data)

print(df1)
```

```
      Name  Age      City
0    Alice   25  New York
1      Bob   30    London
2  Charlie   35     Paris
```

In [6]:

```python
data_rows = [
    {'Name': 'Alice', 'Age': 25},
    {'Name': 'Bob', 'Age': 30}
]
df2 = pd.DataFrame(data_rows)

print(df2)
```

```
    Name  Age
0  Alice   25
1    Bob   30
```

Робота з реальними файлами

In [7]:

```python
try:
    cars_df = pd.read_csv('cars.csv')
    print("cars.csv successfully found")
except FileNotFoundError:
    print("cars.csv not found")
```

```
cars.csv successfully found
```

In [8]:

```python
display(cars_df.head(5))
```

	YEAR	Make	Model	Size	(kW)	Unnamed: 5	TYPE	CITY (kWh/100 km)	HWY (kWh/100 km)	COMB (kWh/100 km)	CITY (Le/100 km)	HWY (Le/100 km)	COMB (Le/100 km)	(g/km)	RATING	(km)	TIME (h)
0	2012	MITSUBISHI	i-MiEV	SUBCOMPACT	49	A1	B	16.9	21.4	18.7	1.9	2.4	2.1	0	NaN	100	7
1	2012	NISSAN	LEAF	MID-SIZE	80	A1	B	19.3	23.0	21.1	2.2	2.6	2.4	0	NaN	117	7
2	2013	FORD	FOCUS ELECTRIC	COMPACT	107	A1	B	19.0	21.1	20.0	2.1	2.4	2.2	0	NaN	122	4
3	2013	MITSUBISHI	i-MiEV	SUBCOMPACT	49	A1	B	16.9	21.4	18.7	1.9	2.4	2.1	0	NaN	100	7
4	2013	NISSAN	LEAF	MID-SIZE	80	A1	B	19.3	23.0	21.1	2.2	2.6	2.4	0	NaN	117	7

In [9]:

```python
cars_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 53 entries, 0 to 52
Data columns (total 17 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   YEAR               53 non-null     int64
 1   Make               53 non-null     str
 2   Model              53 non-null     str
 3   Size               53 non-null     str
 4   (kW)               53 non-null     int64
 5   Unnamed: 5         53 non-null     str
 6   TYPE               53 non-null     str
 7   CITY (kWh/100 km)  53 non-null     float64
 8   HWY (kWh/100 km)   53 non-null     float64
 9   COMB (kWh/100 km)  53 non-null     float64
 10  CITY (Le/100 km)   53 non-null     float64
 11  HWY (Le/100 km)    53 non-null     float64
 12  COMB (Le/100 km)   53 non-null     float64
 13  (g/km)             53 non-null     int64
 14  RATING             19 non-null     float64
 15  (km)               53 non-null     int64
 16  TIME (h)           53 non-null     int64
dtypes: float64(7), int64(5), str(5)
memory usage: 9.0 KB
```

In [10]:

```python
display(cars_df.describe())
```

	YEAR	(kW)	CITY (kWh/100 km)	HWY (kWh/100 km)	COMB (kWh/100 km)	CITY (Le/100 km)	HWY (Le/100 km)	COMB (Le/100 km)	(g/km)	RATING	(km)	TIME (h)
count	53.000000	53.000000	53.00000	53.000000	53.000000	53.000000	53.000000	53.000000	53.0	19.0	53.000000	53.000000
mean	2014.735849	190.622642	19.64717	21.633962	20.541509	2.207547	2.422642	2.301887	0.0	10.0	239.169811	8.471698
std	1.227113	155.526429	3.00100	1.245753	1.979455	0.344656	0.143636	0.212576	0.0	0.0	141.426352	2.991036
min	2012.000000	35.000000	15.20000	18.800000	16.800000	1.700000	2.100000	1.900000	0.0	10.0	100.000000	4.000000
25%	2014.000000	80.000000	17.00000	20.800000	18.700000	1.900000	2.300000	2.100000	0.0	10.0	117.000000	7.000000
50%	2015.000000	107.000000	19.00000	21.700000	20.000000	2.100000	2.400000	2.200000	0.0	10.0	135.000000	8.000000
75%	2016.000000	283.000000	22.40000	22.500000	22.100000	2.500000	2.500000	2.500000	0.0	10.0	402.000000	12.000000
max	2016.000000	568.000000	23.90000	23.300000	23.600000	2.700000	2.600000	2.600000	0.0	10.0	473.000000	12.000000

In [11]:

```python
cars_df.to_csv('processed_cars.csv', index=False)
print ("processed_cars.csv successfully saved")
try:
    cars_df.to_parquet('processed_cars.parquet', index=False)
    print("processed_cars.parquet successfully saved")
except ImportError:
    print("processed_cars.parquet save error")
```

```
processed_cars.csv successfully saved
processed_cars.parquet successfully saved
```

Дослідження даних та маніпуляція над ними

In [12]:

```python
print(f"Columns: {cars_df.columns.tolist()}\n")
print(f"Index: {cars_df.index}\n")
print(f"Shape: {cars_df.shape}\n")
```

```
Columns: ['YEAR', 'Make', 'Model', 'Size', '(kW)', 'Unnamed: 5', 'TYPE', 'CITY (kWh/100 km)', 'HWY (kWh/100 km)', 'COMB (kWh/100 km)', 'CITY (Le/100 km)', 'HWY (Le/100 km)', 'COMB (Le/100 km)', '(g/km)', 'RATING', '(km)', 'TIME (h)']

Index: RangeIndex(start=0, stop=53, step=1)

Shape: (53, 17)
```

In [13]:

```python
display(cars_df.sort_values(by='(kW)', ascending=False).head())
```

	YEAR	Make	Model	Size	(kW)	Unnamed: 5	TYPE	CITY (kWh/100 km)	HWY (kWh/100 km)	COMB (kWh/100 km)	CITY (Le/100 km)	HWY (Le/100 km)	COMB (Le/100 km)	(g/km)	RATING	(km)	TIME (h)
49	2016	TESLA	MODEL S P85D/P90D	FULL-SIZE	568	A1	B	23.4	21.5	22.5	2.6	2.4	2.5	0	10.0	407	12
50	2016	TESLA	MODEL S P90D (Refresh)	FULL-SIZE	568	A1	B	22.9	21.0	22.1	2.6	2.4	2.5	0	10.0	435	12
52	2016	TESLA	MODEL X P90D	SUV - STANDARD	568	A1	B	23.6	23.3	23.5	2.7	2.6	2.6	0	10.0	402	12
33	2015	TESLA	MODEL S P85D/P90D	FULL-SIZE	515	A1	B	23.4	21.5	22.5	2.6	2.4	2.5	0	NaN	407	12
46	2016	TESLA	MODEL S 70D	FULL-SIZE	386	A1	B	20.8	20.6	20.7	2.3	2.3	2.3	0	10.0	386	12

In [14]:

```python
display(cars_df.sort_index().head())
```

	YEAR	Make	Model	Size	(kW)	Unnamed: 5	TYPE	CITY (kWh/100 km)	HWY (kWh/100 km)	COMB (kWh/100 km)	CITY (Le/100 km)	HWY (Le/100 km)	COMB (Le/100 km)	(g/km)	RATING	(km)	TIME (h)
0	2012	MITSUBISHI	i-MiEV	SUBCOMPACT	49	A1	B	16.9	21.4	18.7	1.9	2.4	2.1	0	NaN	100	7
1	2012	NISSAN	LEAF	MID-SIZE	80	A1	B	19.3	23.0	21.1	2.2	2.6	2.4	0	NaN	117	7
2	2013	FORD	FOCUS ELECTRIC	COMPACT	107	A1	B	19.0	21.1	20.0	2.1	2.4	2.2	0	NaN	122	4
3	2013	MITSUBISHI	i-MiEV	SUBCOMPACT	49	A1	B	16.9	21.4	18.7	1.9	2.4	2.1	0	NaN	100	7
4	2013	NISSAN	LEAF	MID-SIZE	80	A1	B	19.3	23.0	21.1	2.2	2.6	2.4	0	NaN	117	7

In [15]:

```python
cars_df.loc[0, 'YEAR'] = 2025

display(cars_df.head(1))
```

	YEAR	Make	Model	Size	(kW)	Unnamed: 5	TYPE	CITY (kWh/100 km)	HWY (kWh/100 km)	COMB (kWh/100 km)	CITY (Le/100 km)	HWY (Le/100 km)	COMB (Le/100 km)	(g/km)	RATING	(km)	TIME (h)
0	2025	MITSUBISHI	i-MiEV	SUBCOMPACT	49	A1	B	16.9	21.4	18.7	1.9	2.4	2.1	0	NaN	100	7

In [16]:

```python
cars_df.loc[0, 'YEAR'] = 2012

display(cars_df.head(1))
```

	YEAR	Make	Model	Size	(kW)	Unnamed: 5	TYPE	CITY (kWh/100 km)	HWY (kWh/100 km)	COMB (kWh/100 km)	CITY (Le/100 km)	HWY (Le/100 km)	COMB (Le/100 km)	(g/km)	RATING	(km)	TIME (h)
0	2012	MITSUBISHI	i-MiEV	SUBCOMPACT	49	A1	B	16.9	21.4	18.7	1.9	2.4	2.1	0	NaN	100	7

In [17]:

```python
temp_df = cars_df.drop(columns=['RATING', 'Unnamed: 5'])

display(temp_df.head(1))
```

	YEAR	Make	Model	Size	(kW)	TYPE	CITY (kWh/100 km)	HWY (kWh/100 km)	COMB (kWh/100 km)	CITY (Le/100 km)	HWY (Le/100 km)	COMB (Le/100 km)	(g/km)	(km)	TIME (h)
0	2012	MITSUBISHI	i-MiEV	SUBCOMPACT	49	B	16.9	21.4	18.7	1.9	2.4	2.1	0	100	7

In [18]:

```python
temp_df = temp_df.drop(index=0)

display(temp_df.head(1))
```

	YEAR	Make	Model	Size	(kW)	TYPE	CITY (kWh/100 km)	HWY (kWh/100 km)	COMB (kWh/100 km)	CITY (Le/100 km)	HWY (Le/100 km)	COMB (Le/100 km)	(g/km)	(km)	TIME (h)
1	2012	NISSAN	LEAF	MID-SIZE	80	B	19.3	23.0	21.1	2.2	2.6	2.4	0	117	7

Доступ до даних та їх індексація

In [19]:

```python
makes = cars_df['Make']

print(makes)
```

```
0     MITSUBISHI
1         NISSAN
2           FORD
3     MITSUBISHI
4         NISSAN
5          SMART
6          SMART
7          TESLA
8          TESLA
9          TESLA
10         TESLA
11     CHEVROLET
12          FORD
13    MITSUBISHI
14        NISSAN
15         SMART
16         SMART
17         TESLA
18         TESLA
19         TESLA
20           BMW
21     CHEVROLET
22          FORD
23           KIA
24    MITSUBISHI
25        NISSAN
26         SMART
27         SMART
28         TESLA
29         TESLA
30         TESLA
31         TESLA
32         TESLA
33         TESLA
34           BMW
35     CHEVROLET
36          FORD
37           KIA
38    MITSUBISHI
39        NISSAN
40        NISSAN
41         SMART
42         SMART
43         TESLA
44         TESLA
45         TESLA
46         TESLA
47         TESLA
48         TESLA
49         TESLA
50         TESLA
51         TESLA
52         TESLA
Name: Make, dtype: str
```

In [20]:

```python
subset = cars_df[['YEAR', 'Make', 'Model']]

print(subset)
```

```
    YEAR        Make                            Model
0   2012  MITSUBISHI                           i-MiEV
1   2012      NISSAN                             LEAF
2   2013        FORD                   FOCUS ELECTRIC
3   2013  MITSUBISHI                           i-MiEV
4   2013      NISSAN                             LEAF
5   2013       SMART  FORTWO ELECTRIC DRIVE CABRIOLET
6   2013       SMART      FORTWO ELECTRIC DRIVE COUPE
7   2013       TESLA         MODEL S (40 kWh battery)
8   2013       TESLA         MODEL S (60 kWh battery)
9   2013       TESLA         MODEL S (85 kWh battery)
10  2013       TESLA              MODEL S PERFORMANCE
11  2014   CHEVROLET                         SPARK EV
12  2014        FORD                   FOCUS ELECTRIC
13  2014  MITSUBISHI                           i-MiEV
14  2014      NISSAN                             LEAF
15  2014       SMART  FORTWO ELECTRIC DRIVE CABRIOLET
16  2014       SMART      FORTWO ELECTRIC DRIVE COUPE
17  2014       TESLA         MODEL S (60 kWh battery)
18  2014       TESLA         MODEL S (85 kWh battery)
19  2014       TESLA              MODEL S PERFORMANCE
20  2015         BMW                               i3
21  2015   CHEVROLET                         SPARK EV
22  2015        FORD                   FOCUS ELECTRIC
23  2015         KIA                          SOUL EV
24  2015  MITSUBISHI                           i-MiEV
25  2015      NISSAN                             LEAF
26  2015       SMART  FORTWO ELECTRIC DRIVE CABRIOLET
27  2015       SMART      FORTWO ELECTRIC DRIVE COUPE
28  2015       TESLA         MODEL S (60 kWh battery)
29  2015       TESLA         MODEL S (70 kWh battery)
30  2015       TESLA      MODEL S (85/90 kWh battery)
31  2015       TESLA                      MODEL S 70D
32  2015       TESLA                  MODEL S 85D/90D
33  2015       TESLA                MODEL S P85D/P90D
34  2016         BMW                               i3
35  2016   CHEVROLET                         SPARK EV
36  2016        FORD                   FOCUS ELECTRIC
37  2016         KIA                          SOUL EV
38  2016  MITSUBISHI                           i-MiEV
39  2016      NISSAN             LEAF (24 kWh battery)
40  2016      NISSAN             LEAF (30 kWh battery)
41  2016       SMART  FORTWO ELECTRIC DRIVE CABRIOLET
42  2016       SMART      FORTWO ELECTRIC DRIVE COUPE
43  2016       TESLA         MODEL S (60 kWh battery)
44  2016       TESLA         MODEL S (70 kWh battery)
45  2016       TESLA      MODEL S (85/90 kWh battery)
46  2016       TESLA                      MODEL S 70D
47  2016       TESLA                  MODEL S 85D/90D
48  2016       TESLA            MODEL S 90D (Refresh)
49  2016       TESLA                MODEL S P85D/P90D
50  2016       TESLA           MODEL S P90D (Refresh)
51  2016       TESLA                      MODEL X 90D
52  2016       TESLA                     MODEL X P90D
```

In [21]:

```python
print(cars_df.iloc[0])
```

```
YEAR                       2012
Make                 MITSUBISHI
Model                    i-MiEV
Size                 SUBCOMPACT
(kW)                         49
Unnamed: 5                   A1
TYPE                          B
CITY (kWh/100 km)          16.9
HWY (kWh/100 km)           21.4
COMB (kWh/100 km)          18.7
CITY (Le/100 km)            1.9
HWY (Le/100 km)             2.4
COMB (Le/100 km)            2.1
(g/km)                        0
RATING                      NaN
(km)                        100
TIME (h)                      7
Name: 0, dtype: object
```

In [22]:

```python
print(cars_df.iloc[4])
```

```
YEAR                     2013
Make                   NISSAN
Model                    LEAF
Size                 MID-SIZE
(kW)                       80
Unnamed: 5                 A1
TYPE                        B
CITY (kWh/100 km)        19.3
HWY (kWh/100 km)         23.0
COMB (kWh/100 km)        21.1
CITY (Le/100 km)          2.2
HWY (Le/100 km)           2.6
COMB (Le/100 km)          2.4
(g/km)                      0
RATING                    NaN
(km)                      117
TIME (h)                    7
Name: 4, dtype: object
```

In [23]:

```python
print(cars_df.loc[6])
```

```
YEAR                                        2013
Make                                       SMART
Model                FORTWO ELECTRIC DRIVE COUPE
Size                                  TWO-SEATER
(kW)                                          35
Unnamed: 5                                    A1
TYPE                                           B
CITY (kWh/100 km)                           17.2
HWY (kWh/100 km)                            22.5
COMB (kWh/100 km)                           19.6
CITY (Le/100 km)                             1.9
HWY (Le/100 km)                              2.5
COMB (Le/100 km)                             2.2
(g/km)                                         0
RATING                                       NaN
(km)                                         109
TIME (h)                                       8
Name: 6, dtype: object
```

In [24]:

```python
print(cars_df.loc[5, 'Model'])
```

```
FORTWO ELECTRIC DRIVE CABRIOLET
```

In [25]:

```python
np.random.seed(42)
indexes = np.arange(cars_df.shape[0])
np.random.shuffle(indexes)
cars_df.index = indexes
cars_df.sort_index(inplace=True)
```

In [26]:

```python
cars_df[cars_df['Make'].isin(['BMW', 'Audi'])]
```

Out[26]:

	YEAR	Make	Model	Size	(kW)	Unnamed: 5	TYPE	CITY (kWh/100 km)	HWY (kWh/100 km)	COMB (kWh/100 km)	CITY (Le/100 km)	HWY (Le/100 km)	COMB (Le/100 km)	(g/km)	RATING	(km)	TIME (h)
45	2016	BMW	i3	SUBCOMPACT	125	A1	B	15.2	18.8	16.8	1.7	2.1	1.9	0	10.0	130	4
49	2015	BMW	i3	SUBCOMPACT	125	A1	B	15.2	18.8	16.8	1.7	2.1	1.9	0	NaN	130	4

In [27]:

```python
cars_df[cars_df['Model'].str.contains('X')]
```

Out[27]:

	YEAR	Make	Model	Size	(kW)	Unnamed: 5	TYPE	CITY (kWh/100 km)	HWY (kWh/100 km)	COMB (kWh/100 km)	CITY (Le/100 km)	HWY (Le/100 km)	COMB (Le/100 km)	(g/km)	RATING	(km)	TIME (h)
38	2016	TESLA	MODEL X P90D	SUV - STANDARD	568	A1	B	23.6	23.3	23.5	2.7	2.6	2.6	0	10.0	402	12
51	2016	TESLA	MODEL X 90D	SUV - STANDARD	386	A1	B	23.2	22.2	22.7	2.6	2.5	2.6	0	10.0	414	12

Фільтрація та вибірка даних

In [28]:

```python
tesla_cars = cars_df[cars_df['Make'] == 'TESLA']

display(tesla_cars.head())
```

	YEAR	Make	Model	Size	(kW)	Unnamed: 5	TYPE	CITY (kWh/100 km)	HWY (kWh/100 km)	COMB (kWh/100 km)	CITY (Le/100 km)	HWY (Le/100 km)	COMB (Le/100 km)	(g/km)	RATING	(km)	TIME (h)
0	2015	TESLA	MODEL S (60 kWh battery)	FULL-SIZE	283	A1	B	22.2	21.7	21.9	2.5	2.4	2.5	0	NaN	335	10
3	2013	TESLA	MODEL S (60 kWh battery)	FULL-SIZE	270	A1	B	22.2	21.7	21.9	2.5	2.4	2.5	0	NaN	335	10
7	2016	TESLA	MODEL S 85D/90D	FULL-SIZE	386	A1	B	22.0	19.8	21.0	2.5	2.2	2.4	0	10.0	435	12
13	2013	TESLA	MODEL S PERFORMANCE	FULL-SIZE	310	A1	B	23.9	23.2	23.6	2.7	2.6	2.6	0	NaN	426	12
14	2016	TESLA	MODEL S P85D/P90D	FULL-SIZE	568	A1	B	23.4	21.5	22.5	2.6	2.4	2.5	0	10.0	407	12

In [29]:

```python
tesla_2016 = cars_df[(cars_df['Make'] == 'TESLA') & (cars_df['YEAR'] == 2016)]

display(tesla_2016.head())
```

	YEAR	Make	Model	Size	(kW)	Unnamed: 5	TYPE	CITY (kWh/100 km)	HWY (kWh/100 km)	COMB (kWh/100 km)	CITY (Le/100 km)	HWY (Le/100 km)	COMB (Le/100 km)	(g/km)	RATING	(km)	TIME (h)
7	2016	TESLA	MODEL S 85D/90D	FULL-SIZE	386	A1	B	22.0	19.8	21.0	2.5	2.2	2.4	0	10.0	435	12
14	2016	TESLA	MODEL S P85D/P90D	FULL-SIZE	568	A1	B	23.4	21.5	22.5	2.6	2.4	2.5	0	10.0	407	12
18	2016	TESLA	MODEL S (70 kWh battery)	FULL-SIZE	283	A1	B	23.8	23.2	23.6	2.7	2.6	2.6	0	10.0	377	12
20	2016	TESLA	MODEL S 70D	FULL-SIZE	386	A1	B	20.8	20.6	20.7	2.3	2.3	2.3	0	10.0	386	12
22	2016	TESLA	MODEL S (60 kWh battery)	FULL-SIZE	283	A1	B	22.2	21.7	21.9	2.5	2.4	2.5	0	10.0	335	10

In [30]:

```python
mask = (cars_df['Make'] == 'FORD') & (cars_df['YEAR'] == 2014)

print(mask.head())
print(f"Column where the condition is met - 25: {mask.loc[25]}" )
```

```
0    False
1    False
2    False
3    False
4    False
dtype: bool
Column where the condition is met - 25: False
```

In [31]:

```python
mask2 = (cars_df['Make'] == 'CHEVROLET') | (cars_df['YEAR'] == 2016)

print(mask2.head())
```

```
0    False
1     True
2     True
3    False
4    False
dtype: bool
```

In [32]:

```python
brands = ['BMW', 'KIA']
display(cars_df[cars_df['Make'].isin(brands)].head())
```

	YEAR	Make	Model	Size	(kW)	Unnamed: 5	TYPE	CITY (kWh/100 km)	HWY (kWh/100 km)	COMB (kWh/100 km)	CITY (Le/100 km)	HWY (Le/100 km)	COMB (Le/100 km)	(g/km)	RATING	(km)	TIME (h)
2	2016	KIA	SOUL EV	STATION WAGON - SMALL	81	A1	B	17.5	22.7	19.9	2.0	2.6	2.2	0	10.0	149	4
16	2015	KIA	SOUL EV	STATION WAGON - SMALL	81	A1	B	17.5	22.7	19.9	2.0	2.6	2.2	0	NaN	149	4
45	2016	BMW	i3	SUBCOMPACT	125	A1	B	15.2	18.8	16.8	1.7	2.1	1.9	0	10.0	130	4
49	2015	BMW	i3	SUBCOMPACT	125	A1	B	15.2	18.8	16.8	1.7	2.1	1.9	0	NaN	130	4

In [33]:

```python
display(cars_df[cars_df['Model'].str.contains('battery', case=False)].head())
```

	YEAR	Make	Model	Size	(kW)	Unnamed: 5	TYPE	CITY (kWh/100 km)	HWY (kWh/100 km)	COMB (kWh/100 km)	CITY (Le/100 km)	HWY (Le/100 km)	COMB (Le/100 km)	(g/km)	RATING	(km)	TIME (h)
0	2015	TESLA	MODEL S (60 kWh battery)	FULL-SIZE	283	A1	B	22.2	21.7	21.9	2.5	2.4	2.5	0	NaN	335	10
3	2013	TESLA	MODEL S (60 kWh battery)	FULL-SIZE	270	A1	B	22.2	21.7	21.9	2.5	2.4	2.5	0	NaN	335	10
18	2016	TESLA	MODEL S (70 kWh battery)	FULL-SIZE	283	A1	B	23.8	23.2	23.6	2.7	2.6	2.6	0	10.0	377	12
22	2016	TESLA	MODEL S (60 kWh battery)	FULL-SIZE	283	A1	B	22.2	21.7	21.9	2.5	2.4	2.5	0	10.0	335	10
23	2016	NISSAN	LEAF (30 kWh battery)	MID-SIZE	80	A1	B	17.0	20.7	18.6	1.9	2.3	2.1	0	10.0	172	6

Об’єднання та агрегація даних

In [34]:

```python
df_a = pd.DataFrame({'ID': [1, 2], 'Val': ['A1', 'A2']})
df_b = pd.DataFrame({'ID': [3, 4], 'Val': ['B1', 'B2']})

display(df_a)
display(df_b)
```

	ID	Val
0	1	A1
1	2	A2

	ID	Val
0	3	B1
1	4	B2

In [35]:

```python
pd.concat([df_a, df_b], axis=0)
```

Out[35]:

	ID	Val
0	1	A1
1	2	A2
0	3	B1
1	4	B2

In [36]:

```python
pd.concat([df_a, df_b], axis=1)
```

Out[36]:

	ID	Val	ID	Val
0	1	A1	3	B1
1	2	A2	4	B2

In [37]:

```python
left = pd.DataFrame({'key': ['K0', 'K1', 'K1', 'K3'], 'A': ['A0', 'A1', 'A2', 'A3']})
right = pd.DataFrame({'key': ['K0', 'K1', 'K1', 'K2'], 'B': ['B0', 'B1', 'B2', 'B3']})

pd.merge(left, right, on='key', how='inner')
```

Out[37]:

	key	A	B
0	K0	A0	B0
1	K1	A1	B1
2	K1	A1	B2
3	K1	A2	B1
4	K1	A2	B2

In [38]:

```python
avg_kw = cars_df.groupby('Make')['(km)'].mean()

print(avg_kw)
```

```
Make
BMW           130.000000
CHEVROLET     131.000000
FORD          122.000000
KIA           149.000000
MITSUBISHI    100.000000
NISSAN        135.166667
SMART         109.000000
TESLA         393.652174
Name: (km), dtype: float64
```

In [39]:

```python
multi_agg = cars_df.groupby('Make')['(km)'].agg(['mean', 'max', 'count'])

print(multi_agg)
```

```
                  mean  max  count
Make
BMW         130.000000  130      2
CHEVROLET   131.000000  131      3
FORD        122.000000  122      4
KIA         149.000000  149      2
MITSUBISHI  100.000000  100      5
NISSAN      135.166667  172      6
SMART       109.000000  109      8
TESLA       393.652174  473     23
```

Очищення та аналіз даних

In [40]:

```python
print(cars_df.isna().sum())
```

```
YEAR                  0
Make                  0
Model                 0
Size                  0
(kW)                  0
Unnamed: 5            0
TYPE                  0
CITY (kWh/100 km)     0
HWY (kWh/100 km)      0
COMB (kWh/100 km)     0
CITY (Le/100 km)      0
HWY (Le/100 km)       0
COMB (Le/100 km)      0
(g/km)                0
RATING               34
(km)                  0
TIME (h)              0
dtype: int64
```

In [41]:

```python
clean_df = cars_df.dropna()

print(f"Кількість рядків до видалення пустих значень: {cars_df["RATING"].size}")
print(f"Кількість рядків після видалення пустих значень: {clean_df["RATING"].size}")
```

```
Кількість рядків до видалення пустих значень: 53
Кількість рядків після видалення пустих значень: 19
```

In [42]:

```python
filled_df = cars_df.fillna(999)

display(filled_df["RATING"].head())
```

```
0    999.0
1     10.0
2     10.0
3    999.0
4    999.0
Name: RATING, dtype: float64
```

In [43]:

```python
students = pd.DataFrame({
    'student_id': [101, 102, 101],
    'name': ['Alice', 'Bob', 'Alice'],
    'score': [85, 92, 85]
})
print(f"Кількість дублікатів в таблиці: {students.duplicated().sum()}\n")
print(students, "\n")
print(students.drop_duplicates())
```

```
Кількість дублікатів в таблиці: 1

   student_id   name  score
0         101  Alice     85
1         102    Bob     92
2         101  Alice     85

   student_id   name  score
0         101  Alice     85
1         102    Bob     92
```

In [44]:

```python
series_nan = pd.Series([1, np.nan, np.nan, 4])
print(series_nan, "\n")

series_fillna = series_nan.fillna(0)
print(series_fillna, "\n")

series_interpolate = series_nan.interpolate()
print(series_interpolate, "\n")

series_ffill = series_nan.ffill()
print(series_ffill)
```

```
0    1.0
1    NaN
2    NaN
3    4.0
dtype: float64

0    1.0
1    0.0
2    0.0
3    4.0
dtype: float64

0    1.0
1    2.0
2    3.0
3    4.0
dtype: float64

0    1.0
1    1.0
2    1.0
3    4.0
dtype: float64
```

Візуалізація даних

In [45]:

```python
import timeit
```

In [46]:

```python
large_df = pd.DataFrame({'nums': range(1_000_000)})

apply_time = timeit.timeit(
    lambda: large_df['nums'].apply(lambda x: x + 10),
    number=10
)

vector_time = timeit.timeit(
    lambda: large_df['nums'] + 10,
    number=10
)

print("apply:", apply_time)
print("vectorized:", vector_time)
```

```
apply: 7.88015430001542
vectorized: 0.08485809992998838
```

In [47]:

```python
clean_chain = (
    cars_df
    .query("Make == 'TESLA'")
    .assign(power_per_km = lambda df: df['(kW)'] / df['(km)'])
    .sort_values(by='power_per_km', ascending=False)
    [['Make', 'Model', 'power_per_km']]
    .head(3)
)

display(clean_chain)
```

	Make	Model	power_per_km
38	TESLA	MODEL X P90D	1.412935
14	TESLA	MODEL S P85D/P90D	1.395577
28	TESLA	MODEL S P90D (Refresh)	1.305747

In [48]:

```python
df_wide = pd.DataFrame({'City': ['London'], '2022': [15], '2023': [18]})
df_long = df_wide.melt(id_vars='City', var_name='Year', value_name='Temp')

print(df_wide, "\n")
print(df_long)
```

```
     City  2022  2023
0  London    15    18

     City  Year  Temp
0  London  2022    15
1  London  2023    18
```

In [49]:

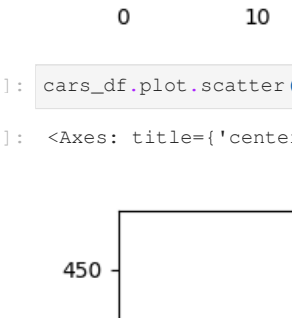
```python
df_dates = pd.DataFrame({'date': ['2024-01-01']})
df_dates['date'] = pd.to_datetime(df_dates['date'])
df_dates['day'] = df_dates['date'].dt.day_name()

print(df_dates)
```

```
        date     day
0 2024-01-01  Monday
```

In [50]:

```python
numeric_df = cars_df[['(kW)', '(km)', 'TIME (h)']]
numeric_df.head().style.background_gradient(cmap='YlGn')
```

Out[50]:



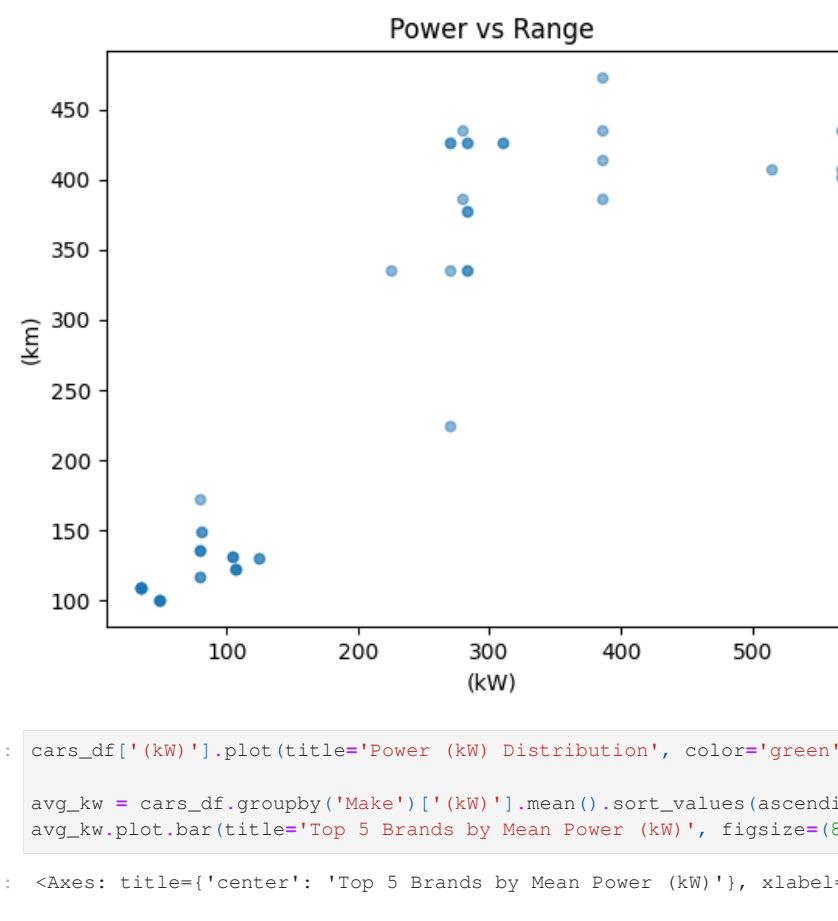
	(kW)	(km)	TIME (h)
0	283	335	10
1	104	131	7
2	81	149	4
3	270	335	10
4	35	109	8

In [51]:

```python
cars_df['(kW)'].plot(title='Power')
```

Out[51]:

```
<Axes: title={'center': 'Power'}>
```

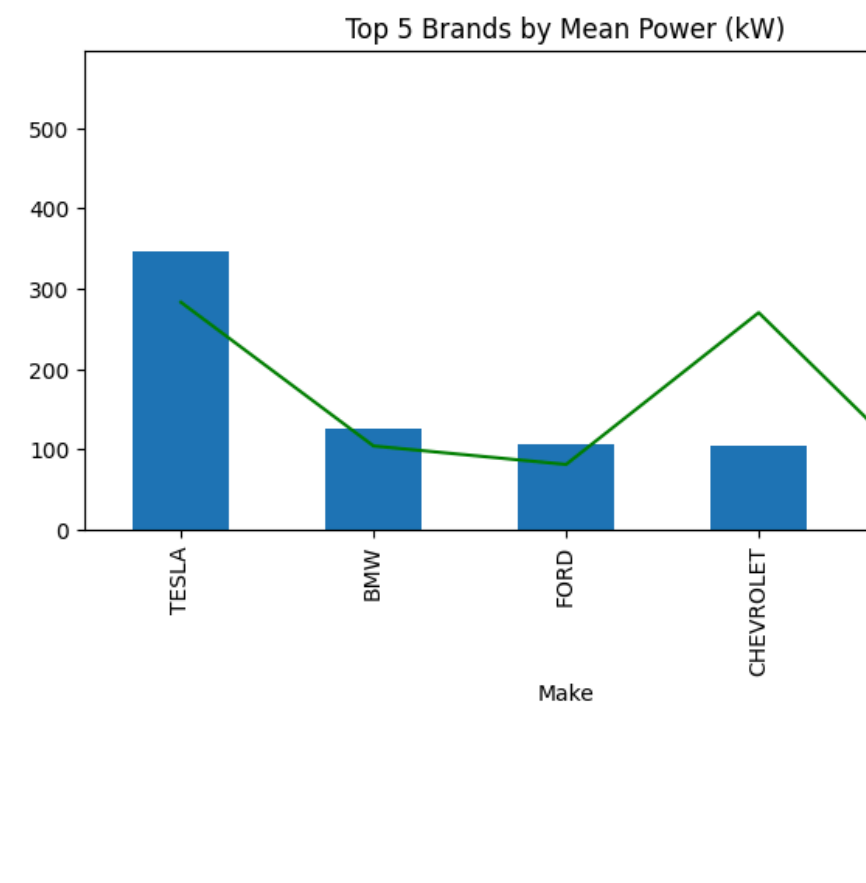


In [52]:

```python
cars_df.plot.scatter(x='(kW)', y='(km)', title='Power vs Range', alpha=0.5)
```

Out[52]:

```
<Axes: title={'center': 'Power vs Range'}, xlabel='(kW)', ylabel='(km)'>
```



In [53]:

```python
cars_df['(kW)'].plot(title='Power (kW) Distribution', color='green')

avg_kw = cars_df.groupby('Make')['(kW)'].mean().sort_values(ascending=False).head(5)
avg_kw.plot.bar(title='Top 5 Brands by Mean Power (kW)', figsize=(8, 4))
```

Out[53]:

```
<Axes: title={'center': 'Top 5 Brands by Mean Power (kW)'}, xlabel='Make'>
```